Assignment 1 - Mazerunner - Report

Adrian Winter - wintea10

Step 1

Why is it preferable to use libraries such as Apache CLI or Log4J to support things like parsing the command line or printing log statements instead of writing code from scratch?

First of all, Apache CLI and Log4J are established libraries that have been thoroughly tested. Their functionality for command line parsing and logging have been proven, which reduces the chance of bugs arising from code written from scratch. These libraries also provide easier troubleshooting through their detailed documentation and community support. Also, using these libraries rather than writing code from scratch allows the programmer to save time on common tasks that have already been solved, and focus more on the core functionality of the application, in this case exploring the maze, ultimately leading to faster development.

What are the advantages/inconveniences of using a logging approach?

The main advantage of logging is debugging and troubleshooting. Maintaining detailed logs helps discover problems in the development of the code by tracking the program execution and changes of variables over time. Logs can help you discover where the error occurs and why. The history of events is also useful for analyzing the behaviour of the code. This is particularly useful in the case where no errors or bugs occur, but the result of the program is different than expected, as you can track the unexpected state change. One inconvenience is the complexity. Loggers can be difficult to configure and use, and in the case of smaller projects such as this one, it would be simpler to implement simple print statements. Another inconvenience is that logging can cause code to become cluttered, making the logic harder to read and maintain. One more inconvenience of logging is that it requires storage and CPU usage, which can lead to increased storage requirements and higher latency.

Step 2

How did you identify these abstractions?

I identified the abstractions by breaking objects down into a specific piece of the overall functionality. For example, Maze object manages maze data, MazeParser deals with converting external data into an internal structure, MazeSolver focuses on exploring the maze, and PathFormatter formats the exploration results. I also considered abstractions that would allow the application to be extended in the future, for example, adding a new algorithm. I also considered how I could isolate the components of the application for easier testing.

What makes you think they are the right ones? Justify your design choices.

For the Maze class, I think this is the correct abstraction because it hides the maze's state and rules from the exploration algorithm. This follows encapsulation and single responsibility principle. The grid data is safely kept hidden from everything else. I think MazeParser was a good choice, as separating the file parsing logic from the maze logic makes the code more modular and easier to maintain or extend. Changes to input format can be isolated to this abstraction without affecting any of the maze logic. The MazeSolverAlgorithm interface provides a method for finding a path and validating a user-path. I think abstracting all maze exploration behaviour as an interface is the correct choice because it allows me to

introduce more than one algorithm in the future if needed, without modifying the rest of the code. This supports the open closed principle, but also isolates all of the maze solving logic into one class (an implementation of the MazeSolverAlgorithm). Overall, I think the separation of concerns, extensibility, maintainability, and testability of my abstractions make them the right choices.

Step 3

How did you identify these features?
I identified these features by starting from the simplest tasks that are necessary to produce the main result of determining a path for the maze, and slowly coming up with more detailed features that ensure every part of the business logic specific is met. For example, I started with very basic features, such as parse the maze, and locate entry/exit points, and then worked my way up to the core features, such as identifying any valid path, and finally, more detailed utility features, such as factorizing the output,

How will you ensure that they model visible value for the end user?
The output from the application, whether it be a path or verification of the user's path will be immediately visible, giving the user tangible results. Additionally, I am going to use loggers up until the final release, which will give the user visible value for features that would otherwise not be visible, such as locating entry and exit points.

Step 4

What made your MVP viable? Justify why your choices make your mvp release a viable one.
My MVP was viable because it was able to parse command line input, read the given file, and create a working maze representation. It proved the viability of maze exploration by showing its accuracy and efficiency in parsing, storing, and representing the maze, without actually computing the exploration. It therefore proved that it could be built upon in future iterations for a maze solving and verifying application.

What made your MVP minimal? Justify why this version is minimal.
My MVP was minimal because it contained the bare minimum features required to fulfill the primary user needs. The implementation was extremely simple, yet it still delivered a working solution by allowing the user to upload and visualize their maze. This allows the user to develop or verify their own path if they need to - therefore solving the core problem, but it does not include any of the unnecessary features of computing/verifying a path for the user, and factorizing the output.

Step 5

How did you encapsulate (information hiding) your maze exploration?
The maze exploration logic is encapsulated within implementations of the MazeSolverAlgorithm interface. Using this encapsulation, the clients rely on abstraction provided by the interface to explore the maze, but the internal decision making process (maze exploration algorithm) is kept hidden. Only the necessary operations of the maze exploration algorithm are exposed. Since the exploration of the maze uses instances, internal state information is also hidden, and only the final result is exposed. Additionally,

the maze exploration logic is kept separate from maze representation, meaning they do not rely on each other to work correctly.

How does it interact with your maze representation?
Since the data of the maze representation is encapsulation, the maze exploration interacts with it through public getters like getGrid, getEntry, and getExit through a public API to obtain the necessary information needed to explore the maze. The maze exploration also uses the public helper method isEmpty from maze, to check whether a cell is a wall, out of bounds, or empty, allowing the exploration to decide where to turn. These abstractions allow the maze exploration to perform its logic without being tied to the specifics of how maze data is stored or constructed. This separation means that if the internal implementation of the maze representation changes, the maze exploration remains unaffected and continues to function in the expected manner.

How is your code design when assessed from a SOLID point of view?
My code is well designed from a SOLID point of view, here are some ways it follows all principles.
Single responsibility principle: Each class is responsible for their own task. Maze is responsible for representing the maze, MazeParser reads and converts a file into a maze structure, RightHandMazeSolver focuses on computing or verifying a path, and Main is responsible for application startup and delegating tasks.
Open closed principle: The common interface of MazeSolverAlgorithm allows for new algorithms to be added without modifying the existing ones. New implementations simply have to implement the interface. This makes the code open for extension but closed for modification.
Liskov substitution principle: Any class that implements the MazeSolverAlgorithm interface can be used interchangeably by the Main class as they all adhere to the same contract. This ensures substituting one algorithm for another does not break the application.
Interface segregation principle: The interface MazeSolverAlgorithm is focused, only including the essential methods for finding and validating a path - which are both required functions for all maze exploration. This ensures that clients are not forced to depend on methods they don't use. If additional functionality were needed later, new interfaces could be defined without affecting core functionality.
Dependency inversion principle: High level components like the Main class depend on abstraction provided by MazeSolverAlgorithm rather than a concrete implementation. This allows the use of different maze exploration algorithms without changing the high level code. Additionally, loose coupling between maze representation and exploration only through public methods means that higher level modules are decoupled from the low level details of maze representation.

How will your code support new algorithms?
The code supports new algorithms through the MazeSolverAlgorithm interface. This interface defines the findAnyPath and validatePath functions. To create a new algorithm you would simply have to create a new implementation of this interface and override the findAnyPath function with one that utilizes the new algorithm. The result of this would leave the original algorithm's implementation unaffected, and you would be able to determine a path using either algorithm from the Main class.

How did iterative and incremental principles manifest in your project?

One way iterative and incremental principles manifested in my project was through continuous refinement. Over time, as new requirements emerged, the code was continuously refactored and improved iteratively. For example, the MazeSolver was refactored to support an interface, which improved encapsulation and also made it easier for the future addition of more algorithms. Continuous refinement also resulted in a more clear separation of concerns, as the division between classes was refined over several iterations, incrementally making my code more modular and maintainable. Also, as a result of frequent testing, issues such as empty lines in the maze file were identified and then fixed incrementally, ensuring a reliable design with no bugs. New functionality was tested at each iteration to confirm that it integrated well with the existing code, allowing me to solve these issues early.

What was the typical timespan of an iteration?
The typical iteration took about one week to complete. Some took less time (half a week), such as parsing data and finding entry and exit points, while other iterations took more time (a week and a half), such as developing the maze exploration algorithm.